

Chapitre 4 : Langage SQL - partie I

1) INTRODUCTION

SQL (**Structured Query Language**, ou **langage de requêtes structurées**) est l'interface externe la plus répandue pour gérer une base de données. C'est un langage déclaratif dans le sens où il suffit d'exprimer ce que l'on veut obtenir, sans devoir expliquer la méthode pour y parvenir. Aujourd'hui, le langage SQL est normalisé et constitue le standard d'accès aux bases de données relationnelles.

SQL est un langage de gestion de bases de données relationnelles. Il étend l'algèbre relationnelle et permet :

- de créer des bases de données
- d'ajouter, modifier et consulter des données d'une base de données existante
- de contrôler les accès aux informations d'une BD

Une instruction SQL peut s'écrire sur plusieurs lignes. Pour être exécutée, l'instruction doit se terminer par un point-virgule. SQL est un langage de requête, pas un langage de programmation. Par conséquent, il n'y a pas de variables ni de structures de contrôles (if, while, for).

SQL se subdivise en 3 sous-langages :

- **LDD** (*Langage de Définition de Données*) : création, modification et suppression des objets que peut manipuler une BD (tables, vues et index, etc.).
- **LMD** (*Langage de Manipulation de Données*) : ajout, suppression, modification et extraction des données.
- **LCD** (*Langage de Contrôle de Données*) : sécurisation et validation des données.

Chacun de ces sous-langages propose ses mots-clés propres. Voici les principales primitives que nous verrons au cours de ce chapitre :

LDD	LMD	LCD
CREATE	SELECT	GRANT
ALTER	INSERT	REVOKE
DROP	UPDATE	COMMIT
	DELETE	ROLLBACK



LANGAGE DE DEFINITION DE DONNEES (LDD)

Le Langage de Définition de Données permet de définir, modifier et supprimer les structures de données, tels que des bases de données, des tables et des vues (LDD ne traite pas les données elles-mêmes).

Les commandes les plus utilisées du LDD :

- **CREATE DATABASE**

La première étape consiste à la création de la base de données avec une commande simple.

Syntaxe :

Pour créer une base de données qui sera appelé « ma_base » il suffit d'utiliser la requête suivante qui est très simple :

```
CREATE DATABASE ma_base
```

Base du même nom qui existe déjà :

Avec MySQL, si une base de données porte déjà ce nom, la requête retournera une erreur. Pour éviter d'avoir cette erreur, il convient d'utiliser la requête suivante pour MySQL :

```
CREATE DATABASE IF NOT EXISTS ma_base
```

L'option IF NOT EXISTS permet juste de ne pas retourner d'erreur si une base du même nom existe déjà. La base de données ne sera pas écrasée.

- **DROP DATABASE**

En SQL, la commande DROP DATABASE permet de supprimer totalement une base de données et tout ce qu'elle contient. Cette commande est à utiliser avec beaucoup d'attention car elle permet de supprimer tout ce qui est inclus dans une base : les tables, les données, les index ...

Syntaxe :

Pour supprimer la base de données « ma_base », la requête est la suivante :

```
DROP DATABASE ma_base
```

Attention : cette commande va supprimer toutes les tables et toutes les données de cette base. Si vous n'êtes pas sûr de ce que vous faites, n'hésitez pas à effectuer une sauvegarde de la base avant de supprimer.

Ne pas afficher d'erreur si la base n'existe pas :



Par défaut, si le nom de base utilisé n'existe pas, la requête retournera une erreur. Pour éviter de maintenir cette erreur si vous n'êtes pas sûr du nom, il est possible d'utiliser l'option IF EXISTS. La syntaxe sera alors la suivante :

```
DROP DATABASE IF EXISTS ma_base
```

• **CREATE TABLE**

La commande CREATE TABLE permet de créer une table en SQL. Un tableau est une entité qui est contenu dans une base de données pour stocker des données ordonnées dans des colonnes. La création d'une table sert à définir les colonnes et le type de données qui seront contenus dans chacun des colonnes (entier, chaîne de caractères, date, valeur binaire ...).

Syntaxe :

La syntaxe générale pour créer une table est la suivante :

```
CREATE TABLE nom_de_la_table (
    colonne1 type_donnees,
    colonne2 type_donnees,
    colonne3 type_donnees,
    colonne4 type_donnees
)
```

Dans cette requête, 4 colonnes ont été définies. Le mot-clé « type_donnees » sera à remplacer par un mot-clé pour définir le type de données (INT, DATE, TEXT ...). Pour chaque colonne, il est également possible de définir si c'est une clé primaire :

- **PRIMARY KEY** : indiquer si cette colonne est considérée comme clé primaire.

Exemple :

```
CREATE TABLE livre (
    numinv varchar(5) primary key,
    titre varchar(20),
    auteur varchar(30),
    quantite int(5)
)
```

• **ALTER TABLE**

La commande ALTER TABLE en SQL permet de modifier une table existante. Il est ainsi possible d'ajouter une colonne, d'en supprimer une ou de modifier une colonne existante, par exemple pour changer le type.



La syntaxe de base :

D'une manière générale, la commande s'utilise de la manière suivante :

```
ALTER TABLE nom_table  
instruction
```

Le mot-clé « instruction » ici sert à désigner une commande supplémentaire, qui sera détaillée ci-dessous selon l'action que l'on souhaite effectuer : ajouter, supprimer ou modifier une colonne.

Ajouter une colonne :

L'ajout d'une colonne dans une table est relativement simple et peut s'effectuer à l'aide d'une requête ressemblant à ceci :

```
ALTER TABLE nom_table  
ADD nom_colonne type_donnees
```

Exemple :

Pour ajouter une colonne qui correspond au nombre de pages sur une table « Livre », il est possible d'utiliser la requête suivante :

```
ALTER TABLE livre  
ADD nbrepages INT(5)
```

Supprimer une colonne :

Une syntaxe permet également de supprimer une colonne pour une table. Il y a 2 manières totalement équivalentes pour supprimer une colonne :

```
ALTER TABLE nom_table  
DROP nom_colonne
```

Ou

```
ALTER TABLE nom_table  
DROP COLUMN nom_colonne
```

Modifier une colonne :

Pour modifier une colonne, comme par exemple changer le type d'une colonne, on utilise la requête suivante :



```
ALTER TABLE nom_table  
MODIFY nom_colonne type_donnees
```

Ici, le mot-clé « type_donnees » est à remplacer par un type de données tel que INT, VARCHAR, TEXT, DATE ...

Renommer une colonne :

Pour renommer une colonne, il convient d'indiquer l'ancien nom de la colonne et le nouveau nom de celle-ci. Il faut également indiquer le type de la colonne.

```
ALTER TABLE nom_table  
CHANGE colonne_ancien_nom colonne_nouveau_nom type_donnees
```

Ici « type_donnees » peut correspondre par exemple à INT, VARCHAR, TEXT, DATE ...

• DROP TABLE

La commande DROP TABLE en SQL permet de supprimer définitivement une table d'une base de données.

Il faut utiliser cette commande avec **attention** car une fois supprimée, les données sont perdues. Avant de l'utiliser sur une base importante il peut être judicieux d'effectuer un backup (une sauvegarde) pour éviter les mauvaises surprises.

Pour supprimer une table « nom_table » il suffit simplement d'utiliser la syntaxe suivante :

```
DROP TABLE nom_table
```

A savoir : s'il y a une dépendance avec une autre table, il est recommandé de la supprimer avant de supprimer la table. C'est le cas par exemple s'il y a des clés étrangères.

Exemple :

```
DROP TABLE Livre
```

L'exécution de cette requête va permettre de supprimer la table « Livre ».



LANGAGE DE MANIPULATION DE DONNEES (LMD)

Le Langage de Manipulation de Données (LMD) permet l'insertion, la recherche, la mise à jour et la suppression de données dans une table de bases de données.

Les quatre commandes principales de ce langage sont : INSERT SELECT UPDATE DELETE.

- **INSERT INTO**

L'insertion de données dans une table s'effectue à l'aide de la commande INSERT INTO. Cette commande permet au choix d'inclure une seule ligne à la base existante ou plusieurs lignes d'un coup.

insérer une seule ligne :

```
INSERT INTO table (nom_colonne_1, nom_colonne_2, ...)  
VALUES ('valeur 1', 'valeur 2', ...)
```

A noter : lorsque le champ à remplir est de type VARCHAR ou TEXT il faut indiquer le texte entre guillemet simple. En revanche, lorsque la colonne est un numérique tel que INT il n'a pas besoin d'utiliser de guillemet, il suffit juste d'indiquer le nombre.

Exemple :

```
INSERT INTO Livre (numinv, titre, auteur, quantite)  
VALUES ('323', 'Bases de donnees', 'C.J.DATE', 3)
```

insérer plusieurs lignes :

Il est possible d'ajouter plusieurs lignes à un tableau avec une seule requête. Pour ce faire, il convient d'utiliser la syntaxe suivante :

```
INSERT INTO table (nom_colonne_1, nom_colonne_2, ...)  
VALUES  
( 'valeur 1', 'valeur 2', ...),  
( 'valeur 3', 'valeur 4', ...),  
( 'valeur 5', 'valeur 6', ... ) ;
```



Exemple : insertion de 4 lignes dans la table « Livre »

```
INSERT INTO Livre (numinv, titre, auteur, quantite)
```

VALUES

```
('323', 'Bases de donnees', 'C.J.DATE',3),
```

```
('124', 'Algorithmes', 'D ;KNUTH',2),
```

```
('1110', 'Programmation', 'B.Meyer',6),
```

```
('241', 'Langage C', 'B.Kernighan',5) ;
```

• SELECT

L'utilisation la plus courante de SQL consiste à lire des données issues de la base de données. Cela s'effectue grâce à la commande **SELECT**, qui retourne des enregistrements dans un tableau de résultat. Cette commande peut sélectionner une ou plusieurs colonnes d'une table.

Obtenir une seule colonne :

L'utilisation basique de cette commande s'effectue de la manière suivante :

```
SELECT nom_du_champ
```

```
FROM nom_du_tableau
```

Cette requête va sélectionner (**SELECT**) le **champ** « nom_du_champ » provenant (**FROM**) du tableau appelé « nom_du_tableau ».

Exemple : Table « Livre » qui contient des informations sur les livres d'une bibliothèque.

NumInv	Titre	Auteur	Quantite
323	Bases De Données	C.J. Date	3
124	Algorithmes	D. Knuth	2
1110	Programmation	B. Meyer	6
241	Langage C	B. Kernighan	5
321	Unix	K. Thompson	2
216	Systèmes Exploitations	Crocus	1
312	Langage Java	J. Gosling	8

Si l'on veut avoir la liste de tous auteurs, il suffit d'effectuer la requête suivante :

```
SELECT Auteur
```

```
FROM Livre
```

Résultat :

Auteur
C.J. Date
D. Knuth
B. Meyer
B. Kernighan
K. Thompson
Crocus
J. Gosling



Obtenir plusieurs colonnes :

Avec la même table « Livre » il est possible de lire plusieurs colonnes à la fois. Il suffit tout simplement de séparer les noms des champs souhaités par une virgule. Pour obtenir les titres et les auteurs des livres il faut alors faire la requête suivante :

```
SELECT titre, auteur
```

```
FROM client
```

Résultat :

Titre	Auteur
Bases De Données	C.J. Date
Algorithmes	D. Knuth
Programmation	B. Meyer
Langage C	B. Kernighan
Unix	K. Thompson
Systèmes Exploitations	Crocus
Langage Java	J. Gosling

Obtenir toutes les colonnes :

Il est possible de retourner automatiquement toutes les colonnes d'un tableau sans avoir à connaître le nom de toutes les colonnes. Au lieu de lister toutes les colonnes, il faut simplement utiliser le caractère « * » (étoile). Il s'utilise de la manière suivante :

```
SELECT *
```

```
FROM Livre
```

Cette requête retourne exactement les mêmes colonnes qu'il y a dans la table. Dans notre cas, le résultat sera donc :

NumInv	Titre	Auteur	Quantite
323	Bases De Données	C.J. Date	3
124	Algorithmes	D. Knuth	2
1110	Programmation	B. Meyer	6
241	Langage C	B. Kernighan	5
321	Unix	K. Thompson	2
216	Systèmes Exploitations	Crocus	1
312	Langage Java	J. Gosling	8